School of Science

EDITH COWAN UNIVERSITY

Programming Principles

Using Git for Assessments

Introduction

This unit requires you to use a program named "Git" to manage your assessment work. Git is a "version control system"; a program that tracks versions of files to help manage projects. It is an industry standard tool used by most software developers. Although the unit's assessments are relatively small single-person projects, understanding the basics of Git is a valuable skill.

This guide and its accompanying videos cover how to install and use Git for this unit's assessments.



Only what you need

This guide *only covers what you need* to use Git to manage your assessment work in this unit. For example, Git's features relating to collaborating as a team and "branching" are *not* covered.

Simple text commands

This guide covers interacting with Git using *text commands*. Although there are graphical interfaces for Git, the text commands are easier, less confusing, and the same across Windows and Mac.

Git vs. GitHub

Software is usually developed by *multiple people who all need access to the project's files*. "GitHub" allows files tracked using Git to be stored online, making them accessible to everyone who needs them. Since this unit's assessments are done individually, you do *not* need GitHub.

Git is not a backup

Git stores earlier versions of the files it is tracking in the same location as the current files, which will not help if you lose access to your computer. *You* are still responsible for backing up your work.

Installing and Configuring Git

Before you start working on your assessments, you must install Git on the computer you are using. While *using* Git is the same on Windows and Mac, initially *installing* it is a different process.

Windows

1. Go to https://git-scm.com/downloads/win and **download** the latest 64-bit version of Git.

2. Run the file you downloaded to open the setup program.

3. Install Git with default settings by clicking "**Next**" through all the options.
❗Although it should not be needed, you may want to change the default editor to Notepad❗

4. On the final screen, tick the "**Launch Git Bash**" box then click "**Finish**".
❗The Git Bash window ("MINGW64") will be used to set your name and email in the next step❗

Mac

There is no package or setup program for Git on MacOS. Instead, use a tool named "Homebrew".

1. Go to https://brew.sh/ and follow the instructions to **install Homebrew**.
❗Homebrew is a "package manager"; a tool that lets you download and install software via text commands❗

2. Once Homebrew is installed, type "**brew install git**" into the Terminal to install Git.

Setting your name and email

At this point, you should have Git installed and have a **command-line window** open: In Windows, this is the Git Bash window opened at the end of the setup, and on Mac it is the Terminal window.

All of the Git commands entered into command-line windows are the same on Windows and Mac.
❗Git commands also work in other command-line windows, such as the Windows command prompt or PowerShell❗

Setting your name and email allows Git to record who is modifying the files it is tracking. Enter the following two commands into your command-line window to **set your name and email address**:

```
git config --global user.name "your name as shown in ECU systems"

git config --global user.email "your ECU email address"
```

Make sure to put your own name and ECU email address between the quote marks!

There will not be any response after entering the commands. If you encounter an error message or type something incorrectly, simply double-check your spelling and try again.

Well done, you have installed and configured Git! You can close the command-line window.

Creating a Repository

A project that Git is tracking the files for is known as a "**repository**". You will need to create a *separate repository for each assessment, before you start working on it*. **Create a folder** that you will use to store the files of an assessment and **open it** in File Explorer (Windows) or Finder (Mac).

Open a **command-line window at the folder's location**:

- In **Windows**, right-click on or inside the folder and select "Open in Terminal".
  ! If you don't see this option, hold Shift then right-click and select "Open PowerShell window here" !
- On a **Mac**, control-click the folder in the path bar and select "Open in Terminal".
  ! If you don't see the path bar at the bottom of the Finder window, choose View > Show Path Bar !

The command-line window should make it clear whether you are in the intended folder.

Once a command-line window is open at the folder, **create a repository** with this command:

```
git init
```

You should see a message confirming that an empty Git repository has been initialised.

Working in a repository

When you create a repository in a folder, Git creates a **hidden subfolder** that it uses to manage things. This is where it keeps copies of earlier versions of files and information about them.

You do not need to look at or interact with the hidden subfolder: You simply *work on the files for your assessment as normal*, and use Git commands via a command-line window when needed.
! Git does not do anything unless you tell it to: It is up to you to "commit" new or changed files to the repository !

Git does *not* change how you make backups of your work or how you send it to staff for help and feedback: You can copy, upload, and email files in the same way that you normally would. If you make changes to a copy of a file outside of your repository, e.g. working on it using a different device, you can simply replace the version in the repository and commit the file as covered below.

Got a question? The final section of this guide contains answers to **Frequently Asked Questions**.

If you are still unsure about anything, please contact unit staff via email or the discussion board!

Committing Changes

Once Git is installed and configured and you have created a repository, you are ready to *start working on the assessment itself*. This involves creating, modifying and saving files using various programs, e.g. Python programs in IDLE and pseudocode in Word, over an extended period of time.

At various points throughout this time, use Git to "commit" the files to the repository. This stores a copy of the files in their current state, along with a note about the changes you've made to them. By doing this, you maintain a navigable historic record of how the files have changed over time.

When to commit

You should ***commit after each significant piece of progress or meaningful change***. You don't want to commit after every little change since that will be tiresome, but you also don't want to commit massive changes involving numerous files or parts of a program since that is too broad.

The exact frequency of your commits is up to you, but here are some suggestions:

- After writing a significant section of pseudocode.
- After implementing a feature or requirement, or a significant part of one.
- After fixing a significant bug or problem with the code.
- After rewriting or redesigning a section of code or pseudocode.
- After a collection of minor changes, e.g. fixing spelling mistakes and formatting.

Your commits should reflect *meaningful changes*, not periods like "every 2 hours" or "end of day".

How to commit

To commit your files to the repository, **open a command-line window in the repository folder** (refer to Page 3 for how to do this), and enter these two commands:

```
git add .
git commit -m "commit message"
```

See below for guidance on writing good commit messages!

The first command makes it so that all files in the folder will be committed, and the second one commits them. You should see a few lines summarising the changes to files after committing.

Commit messages

The message you write when committing should be a *clear and concise summary of the changes* since the last commit. If you cannot describe it in a short sentence, you should commit more often.

Write messages that would be meaningful to someone looking back at the repository in the future.

Use consistent terminology, e.g. "add" for new code, "update" for changes, "fix" for bug fixes, etc. It is standard practice to use imperative mood, e.g. "add" and "fix" rather than "added" and "fixed".

Bundling and Submitting

When you are ready to submit your assessment, the final step is to "bundle" the repository. This packages the files and the hidden Git subfolder of your repository into a single file that you can submit on the unit site, allowing staff to download and "clone" the repository – recreating it on their own computer for marking.

Staff will be able to view the commit history of your repository and see how your work evolved.

Tidying up

First, make sure that your repository folder **only contains files that you want to submit**. Read the instructions in your assessment brief and make sure that you are submitting everything required. **Move or delete any files you do not want to submit**, or that could cause confusion when marking.

It is ok to have extra files in the folder, such as the Word version of your pseudocode PDF, but make sure that it is clear which files should be considered when marking your assessment.

Then, follow the instructions on Page 4 to **commit the final versions of your files**.
❗ Only committed versions of files will be included in a bundle ❗

Bundling a repository

After committing the final versions of your files, enter this command in **a command-line window in the repository folder** to create a bundle of your repository:

```
git bundle create assessment.bundle --all
```

The filename you specify before ".bundle" is not important – it gets renamed when staff download it!

You should see a bunch of messages about enumerating, counting and compressing, and be left with a file matching the name specified in the command in your repository folder.

Submit this file in the appropriate location of the unit site to submit your assessment. The unit site will only allow you to submit a ".bundle" file, and only bundled Git repositories will be marked.
❗ See the FAQ on the next page if you would like to check what your bundle contains before submitting it ❗

Frequently Asked Questions

If you have a question that is not answered in this document, contact the unit coordinator.

I am already using Git/GitHub for other projects; how does this change things?

You may want to set your ECU name/email local to assessment repositories, rather than globally.

Can I use GitHub if I have a specific reason that I need/want to?

Yes. If you use GitHub, any assessment-related repositories *must be (and remain) private* to prevent your work from being plagiarised.

Can I integrate Git with an IDE such as Visual Studio or Xcode?

Yes, but it is up to you to set it up and learn to use it. What you need to submit remains the same.

Why doesn't this guide cover Linux/Unix?

Linux and Unix users are quite rare, and tend not to need a guide for such things.

What are some other useful Git commands?

Use "**git status**" to show which files in a repository are being tracked, ready to be committed, etc.
Use "**git log**" to show the repository's commit history ("**git log --oneline**" for a compact version).
❗If there is lots to show, you may need to scroll down with arrow keys and enter 'q' to exit the log❗

Will my files or repositories be affected if I uninstall or update Git?

No. Your repository folders will not change, and all files will remain. The hidden subfolder also remains, allowing you to interact with the repository using Git commands in the future if needed.

How do I check what my ".bundle" file contains before submitting it?

Move the bundle file to another location and open a command-line window in that location, then enter "**git clone**" followed by the name of the bundle file to unpack it into a new folder/repository.

What if I started or completed an assessment without creating a repository? / What if I forgot to regularly commit changes?

Your submission *must* be a bundled repository to be marked, even if that repository does not have a good commit history. This will impact your mark in the relevant section of the rubric.